

Auth Module API Documentation

Base URL: `/auth` **Module:** Authentication (Phone + Google/Apple OAuth + Token Management)

1. Phone Authentication

1.1 Request OTP

- **Endpoint:** `POST /auth/phone/request-otp`
- **Description:** Sends a one-time password (OTP) to the user's phone.
- **Request Body:**

```
{
  "phone": "+911234567890"
}
```

- **Response:**

```
{
  "message": "OTP sent successfully"
}
```

- **Frontend Flow:**
 - User enters phone number.
 - Frontend calls this endpoint.
 - Backend sends OTP via SMS.
-

1.2 Verify OTP

- **Endpoint:** `POST /auth/phone/verify-otp`
- **Description:** Verifies OTP sent to the user.
- **Request Body:**

```
{
  "phone": "+911234567890",
  "otp": "123456"
}
```

- **Response (Success):**

```
{
  "accessToken": "<JWT_ACCESS_TOKEN>",
  "refreshToken": "<JWT_REFRESH_TOKEN>",
  "user": {
```

```
{
  "id": "uuid",
  "name": "John Doe",
  "email": null,
  "phone": "+911234567890"
}
```

- **Frontend Flow:**

- User enters OTP.
 - Frontend sends OTP + phone number.
 - Backend verifies OTP and issues tokens.
-

2. Token Management

2.1 Refresh Tokens

- **Endpoint:** POST /auth/refresh

- **Description:** Generates new access and refresh tokens using a valid refresh token.

- **Request Body:**

```
{
  "refreshToken": "<JWT_REFRESH_TOKEN>"
}
```

- **Response:**

```
{
  "accessToken": "<NEW_JWT_ACCESS_TOKEN>",
  "refreshToken": "<NEW_JWT_REFRESH_TOKEN>"
}
```

- **Frontend Flow:**

- When access token expires, frontend calls this endpoint with the refresh token.
 - Backend returns new tokens.
-

2.2 Logout

- **Endpoint:** POST /auth/logout

- **Description:** Logs out the user by invalidating the refresh token.

- **Request Body:**

```
{
  "refreshToken": "<JWT_REFRESH_TOKEN>"
}
```

- **Response:**

```
{
  "message": "Logged out successfully"
}
```

- **Frontend Flow:**

- User clicks logout → frontend calls endpoint with refresh token → backend invalidates token.
-

3. Google App OAuth

3.1 Google App Login

- **Endpoint:** POST /auth/google/app
- **Description:** Sign in using Google App OAuth.
- **Request Body:**

```
{
  "id_token": "<GOOGLE_ID_TOKEN>"
}
```

- **Response (Success):**

```
{
  "accessToken": "<JWT_ACCESS_TOKEN>",
  "refreshToken": "<JWT_REFRESH_TOKEN>",
  "user": {
    "id": "uuid",
    "name": "Google User",
    "email": "user@gmail.com",
    "phone": null
  }
}
```

- **Frontend Flow:**
 - Integrate Google SDK to get id_token.
 - Send id_token to backend.
 - Backend verifies token and issues JWT tokens.
-

4. Apple App OAuth

4.1 Apple App Login

- **Endpoint:** POST /auth/apple/app
- **Description:** Sign in using Apple App OAuth.
- **Request Body:**

```
{
  "id_Token": "<APPLE_ID_TOKEN>"
}
```

⚠ Pass the token exactly as received from Apple.

• **Response (Success):**

```
{
  "accessToken": "<JWT_ACCESS_TOKEN>",
  "refreshToken": "<JWT_REFRESH_TOKEN>",
  "user": {
    "id": "uuid",
    "name": "Apple User",
    "email": "user@apple.com",
    "phone": null
  }
}
```

• **Frontend Flow:**

- Integrate Apple Sign-In SDK.
- Send `id_Token` to backend.
- Backend verifies token, creates/validates user, issues tokens.

5. Authentication Flow (Summary)

Phone Auth:

[User enters phone] --> POST /request-otp --> OTP sent

[User enters OTP] --> POST /verify-otp --> accessToken + refreshToken

Google OAuth:

[User signs in with Google SDK] --> POST /google/app --> backend verifies --> accessToken + refreshToken

Apple OAuth:

[User signs in with Apple SDK] --> POST /apple/app --> backend verifies --> accessToken + refreshToken

Token Refresh:

[Access token expired] --> POST /refresh --> new tokens

Logout:

[User clicks logout] --> POST /logout --> refresh token invalidated

6. Frontend Notes

- All endpoints are **POST** and expect **JSON** bodies.
 - Store `accessToken` securely (memory/session) and `refreshToken` in a secure storage/cookie.
 - Include `Authorization: Bearer <accessToken>` for protected API calls.
 - Phone authentication may require SMS permissions.
 - Google/Apple OAuth requires SDK integration.
-

✓ This document can be used directly by frontend developers for integration.